

ELTE EÖTVÖS LORÁND UNIVERSITY

VISUALIZATION AND CONTENT MANAGEMENT WITH ASP.NET MVC

Modern web application development in .NET

Lecture 2

Dr. Máté Cserép

mcserep@inf.elte.hu

https://mcserep.web.elte.hu/

VIEWS

ASP.NET supports multiple description languages for defining views

- Razor has the simplest syntax and is the most popular;
- The view can be strongly typed, in which case the viewmodel type is specified, e.g. **`@model MyProject.Model.ItemModel`**
- Dynamic items are marked with the @ prefix
  - the block **`@{ ... }`** can contain arbitrary view logic in C#;
  - the block **`@* ... *@`** denotes multiline comments;
  - conditional (**`@if`**, **`@else`**) and iterative (**`@for`**, **`@foreach`**, **`@while`**) execution can be used;
  - namespace usage can be resolved with **`@using`** (otherwise canonical paths must be used).

VIEWS

- In the view the viewmodel can be referenced by the **`Model`** element
- The special **`Html`** static class is available as a HTML helper, e.g.:
  - hyperlinks to actions (**`ActionLink`**), in which we specify the action, the controller and the arguments (usually as an anonymous object), e.g.:

```
Html.ActionLink("Load image", "LoadImage",
                "Home", new {id = 1}, null);
                // Home/LoadImage?id=1 link
```

  - forms (**`BeginForm`**, **`EndForm`**)
  - display and input elements (**`LabelFor`**, **`TextBoxFor`**, **`PasswordFor`**), validators (**`ValidationMessageFor`**) for forms
  - unencoded content (**`Raw`**)

VIEWS

- The dynamic user controls can also be defined with *tag helpers*, utilizing special attributes marked with **`asp-`** prefix:
  - Linking an action:
    ```
    <a asp-controller="Home" asp-action="Index">Home</a>
    ```
  - Embed a textbox for the given property of the viewmodel:
    ```
    <input asp-for="Name">
    ```
  - Embed client side script files: `<script src="~/js/site.js" asp-append-version="true"></script>`
    - Appends the **`v=<hash>`** query parameter to the URL, for client side cache invalidation of the loaded file in the browser, in case of content change.
  - Embed a stylesheet: `<link rel="stylesheet" href="~/css/site.min.css" asp-append-version="true" />`

VIEWS

- More specific paths can be handled by the Url class, e.g.:
  `Url.Content("~/style.css")`
  `Url.Action("Action", "Controller")`
- Views can depend on layouts and customized for different profiles (e.g. desktop/mobile environment)
- In ASP.NET MVC the view is an object that implements the **`IView`** interface and the view's descriptive language (*the engine*) implements the **`IViewEngine`** interface
  - It is supported to implement new engines and views by anyone

VIEWS

Beside the *strongly typed* viewmodel, we can pass data and even functions (as *lambda objects*) to the view through the special **`ViewBag`** property, available both in the controller and the view

- the **`ViewBag`** is a dynamic object, meaning it is dynamically managed and *weakly typed* (runtime type checking);
- it is also an **`ExpandoObject`**, i.e. any property or method can be assigned to it, so it can take any value, e.g.:
  `ViewBag.Message = "Hello";`
- the contents is available in the view as well, e.g.:
  `<div>@ViewBag.Message</div>`
- the **`ViewData`** can be used for the same purpose, but as a dictionary.

VIEWS

E.g. (MyController.cs):

```
public class MyController : Controller {
  // ...
  public IActionResult List() {
    ViewBag.Title = "List of names";
    // set the title of the page
    String[] itemNames = …;
    return View("ListPage", itemNames);
    // pass a string array to the ListPage view
  }

  public IActionResult Details(String name){
    return View("DetailsPage", …);
  }
}
```

VIEWS

E.g. (ListPage.cshtml):

```
@model IEnumerable<String>
@* the viewmodel is strongly typed *@
@{ Layout = null; // no layout used }

<!DOCTYPE html>
<html>
  <head>
    <title>@ViewBag.Title</title>
  </head> @* the title is defined in the controller *@
  <body>
```

VIEWS

E.g. (ListPage.cshtml):

```
    @if (Model != null){ // conditional execution
      <div>
        @foreach (String name in Model){
          <span><b>@name</b>
          <a asp-action="Details" asp-route-name="@name">See details</a>
          </span> @* link to the Details action *@
        }
      </div>
    } else {
      <div>No items found!</div>
    }
  </body>
</html>
```

LAYOUTS AND PARTIAL VIEWS

In many cases, our view is made up of various parts

- some parts (e.g. headings, menus) appear on more than one page, others are specific to a single or certain pages
- recurring parts give our website a consistent look and experience

Repetitive content can be extracted and reused in multiple views

- these parts are therefore not necessarily associated with a single controller, but are *shared* between controllers (*shared view*) and placed in the **`Views/Shared`** directory

PARTIAL VIEWS

A *partial view* is a view that does not render the whole page, but only a part, a segment of it

- this content can be displayed in another view using the **`Html.RenderPartial()`** statement
  - we can specify the name of the partial view, the passed strongly typed viewmodel and even a customized viewdata
- e.g. (**`Views/Shared/Item.cshtml`**):

```
@model string @* the viewmodel is single string item *@

<span>Item @ViewBag.Index/@ViewBag.Count:
      <b>@Model</b>
</span>
```

PARTIAL VIEWS

- E.g. for using this partial view (**MyView.cshtml**):

```
@model List<string> @* the viewmodel is a string list *@
@{ int idx = 1; } @* set an index counter *@

<div>
  @foreach(string item in Model) {
    @Html.RenderPartial("Item", @* render Views/Shared/Item.cshtml *@
      item, @* pass the current item *@
      new ViewDataDictionary(ViewData) {
        { "index", idx }, { "count", Model.Count }
      }
    ) @* pass the index and the length of the list *@
  }
</div>
```

PARTIAL VIEWS

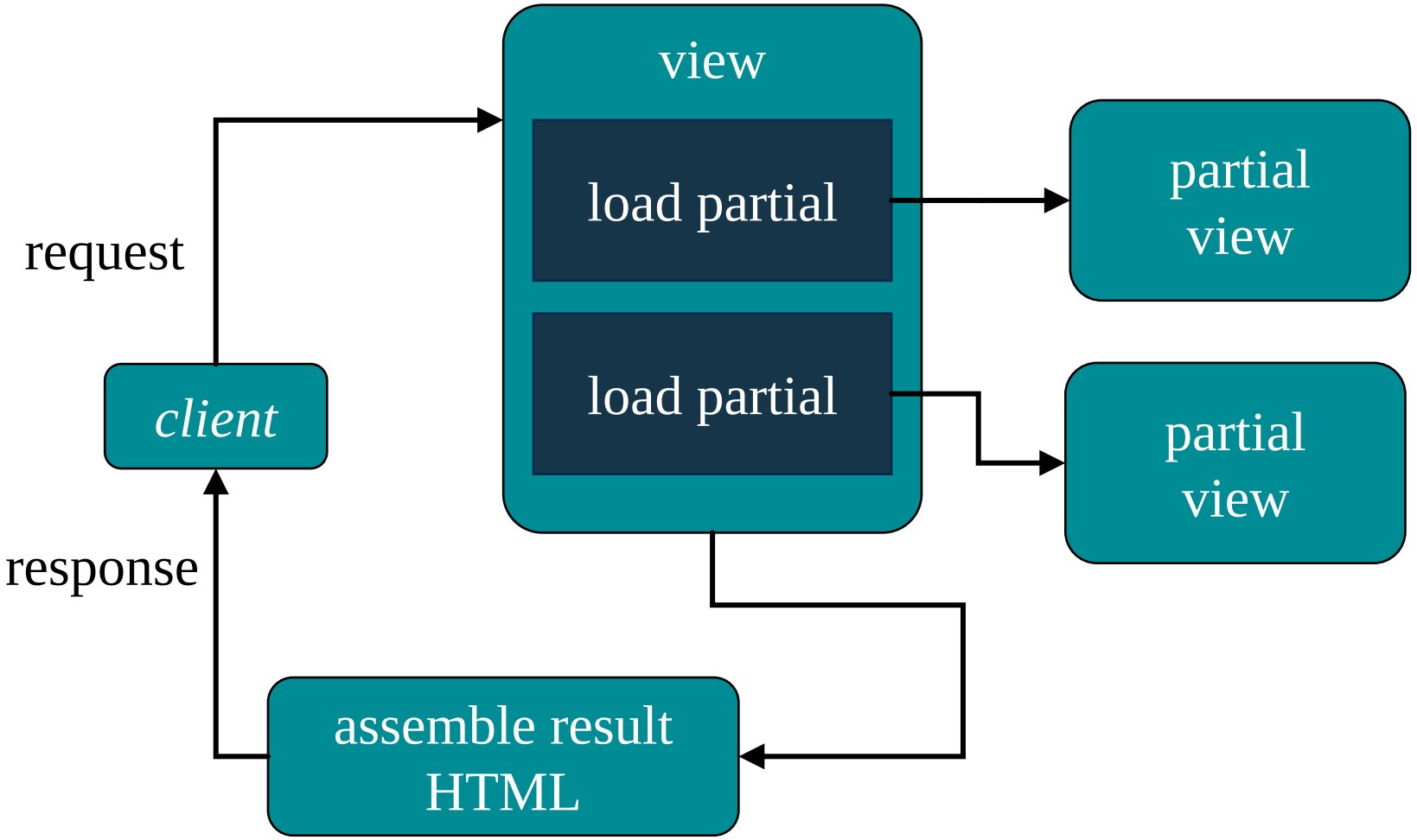
The tag helper syntax offers an alternative here as well, e.g.:

```
@model List<string> @* the viewmodel is a string list *@
@{ int idx = 1; } @* set an index counter *@

@{
  var viewData = new ViewDataDictionary(this.ViewData) {
    { "index", idx }, { "count", Model.Count }
  };
} @* extend the viewdata with a new property *@

@foreach (String name in Model) {
  <partial name="Item" model="name" view-data="viewData" />
} @* pass the current item and the extended viewdata *@
```

PARTIAL VIEWS



PARTIAL VIEWS

A *partial view* can be returned by controller action as well, with the inherited **`PartialView`** method, e.g.:

```
public class ProductController : Controller {
  ...
  // PartivalViewResult is an IActionResult
  public PartivalViewResult Details(int id) {
    Item item = …
    return PartialView("Details", item);
  }
}
```

This is still reusable in other views with the **`Html.RenderAction()`** helper method.

LAYOUT VIEWS

Layouts allows us to specify multiple replaceable content areas (placeholders) within a page, for which the actual can be loaded from other views

- the layout view is the frame that defines the fixed content
- the replaceable contents are the *sections* defined in each view
  - sections are specified using the **`@section <name> { ... }`** block
  - the body is a special section, which is not marked with a distinct section name
  - in our views we must specify the layout we want to use with the **`Layout`** property

LAYOUT VIEWS

E.g. (**View.cshtml**):

```
@model ProductsModel
@{ Layout = "~/Views/Shared/_Layout.cshtml;
// we use a layout
}
@section mainMenu { // section defining our main menu
  foreach (String item in Model.ItemTypes)
    <div>@Html.ActionLink(…)</div>
  }
@* content not within a section is the body *@
<div id="title">List of Products</div>
...
@section mainFooter { ... } // another section
```

LAYOUT VIEWS

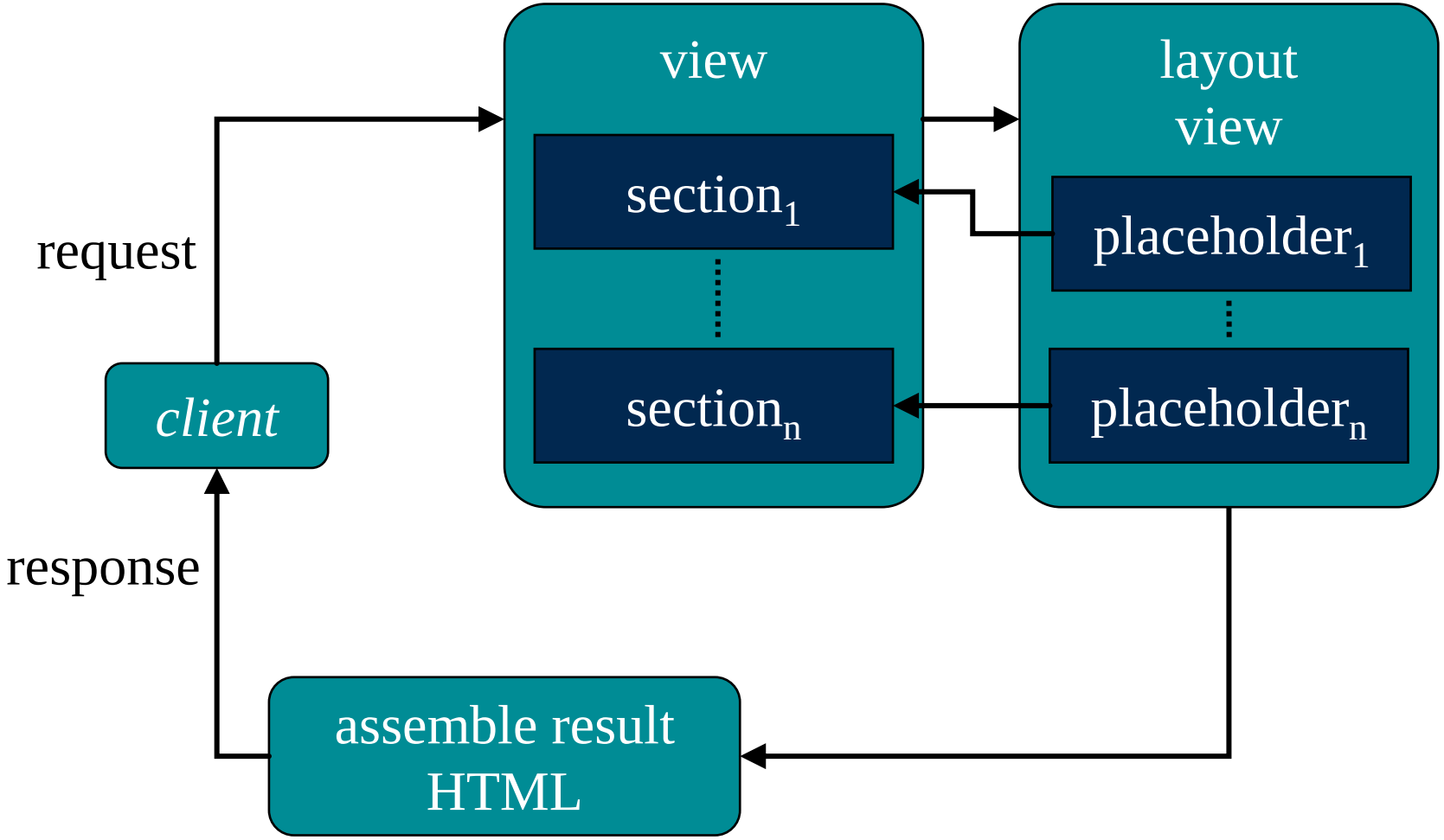
- In the layout view, the body is loaded by the **`RenderBody()`** call, the other sections by the **`RenderSection(<name>)`** call
  - if a section is not required to be defined, the false value of the **`required`** parameter can be used to indicate this;
  - the existence of the section can be checked by **`IsSectionDefined(<name>)`**.
- The file name of the layout view is conventionally started with an underscore, the default layout is **`Views/Shared/_Layout.cshtml`**
  - this does not require the **`Layout`** property;
  - this is controlled by the **`Views/_ViewStart.cshtml`** file.
- Layouts can be nested.

LAYOUT VIEWS

E.g.:

```
<body>
  ... @* other content *@
  <div id="menu">
    @RenderSection("mainMenu")
    @* load section *@
  </div>
  <div id="mainpage">
    @RenderBody() @* load body *@
  </div>
  <div id="footer">
    @RenderSection("mainFooter", false)
    @* load optional section *@
  </div>
</body>
```

LAYOUT VIEWS



FILE RESULT

We may send not only HTML content, but any file content (e.g. images, documents, compressed files) to the user

- the content has a type (*internet media type*, *MIME type*)
- if the browser can display the received type, it can display it, otherwise it can offer it for download
- file content can be returned using the **`File`** helper method inherited
  - the content can be defined as a binary content, a stream, or path;
  - the MIME type must be specified;
  - we can optionally specify the name of the downloaded file (then the file will be offered for download regardless of the type).

FILE RESULT

E.g.:

```
return File("/Content/Pictures/image.jpg", "image/jpg"); //
load and return a JPEG image from the file system

Byte[] imageFile = … // load an image from the database
return File(imageFile, "image/png", "image.png");
// return PNG image for download with the name image.png
```

- The returned file content can be displayed in a view with the **<img>** HTML tag using the **Url.Action** helper method to create the URL.

LAYOUTS AND PARTIAL VIEWS

Layouts should be used when:

- you want to define the structure of your page (e.g. menu, header)
- you want to display the same elements in the same way on several pages

Partial views should be when:

- you want to use the same elements in various environments (e.g. login box), or you want to make a particular element interchangeable (e.g. table/chart)
- you want to replace or reload only a part of the page



ELTE EÖTVÖS LORÁND UNIVERSITY

DATABASE MANAGEMENT WITH .NET CORE

Modern web application development in .NET

DATABASE MANAGEMENT

Relational database management systems (*RDBMS*) store data in *tables*

- Tables have columns (*fields*) and rows (*records*)
- The definition of the table structure is called a *schema*

For fields, *keys* can be defined to specify constraints (e.g. unique occurrences) and to increase the efficiency of query operations (*indexing*)

- Tables may have a *primary key* that specifies the primary, physical order of the data

Relationships between tables can be defined using *foreign keys* with optionally *reference integrity checks* enforced

DATABASE MANAGEMENT

Microsoft has its own SQL database management solution, *Microsoft SQL Server* (*MSSQL*)

- SQL Server Management Studio is a widely used, free client tool
  - *Visual Studio itself* can also be used (*View/Server Explorer*, *View/SQL Server Object Explorer*, *Tools/Sql Server*), with limited capabilities
- has its own dialect language (*Transact-SQL*), which is compatible with the SQL standard
  - includes some special statements/types, e.g. automated increment feature is supported through the **`IDENTITY`** statement
- user management supports individual accounts as well as Windows-based authentication

DATABASE MANAGEMENT

The *connection string* defines the parameters of the database connection

- usually contains the name or IP address of the server, the name of the database, and further connection data (username/password)
- the exact content varies from one database manager to another
- e.g. (SQL Server with standard security):
  ```
  "Server=localhost;Database=myDataBase;User Id=myUser;Password=myPassword;"
  ```
- e.g. (PostgreSQL with Windows authentication):
  ```
  "Server=127.0.0.1;Port=5432;Database=myDataBase; Integrated Security=true;"
  ```
- Reference: www.connectionstrings.com

DATABASE MANAGEMENT

There are multiple solutions to manage databases in the .NET Core framework

- *native connection*: execute SQL statements directly on the physical database
- *logical relational model*: represents the physical structure and of the relational database in the memory (tables, columns, rows, relations)
- *entity-based object relational model* (*Entity Framework*): maps the database schema and the contained data into an object-oriented structure in a parameterized manner

We will focus on using the Entity Framework, as the modern approach!

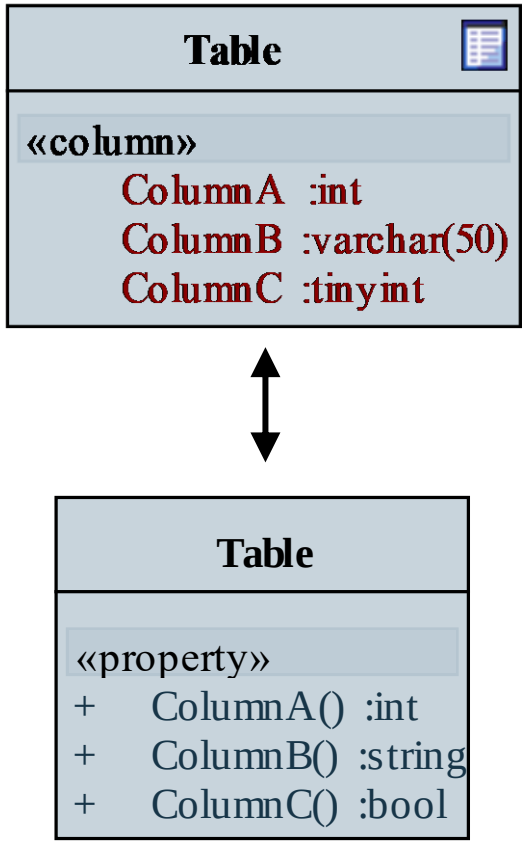
OBJECT-RELATIONAL MAPPING

Data management programs are usually built in an object-oriented way, therefore the data management should also be done this way

In relational databases:

- data is grouped into tables, which defines the schema, the way the data is structured, i.e. the *type*
- a row stores the data for a given element, i.e. the row is an *instance* of the type

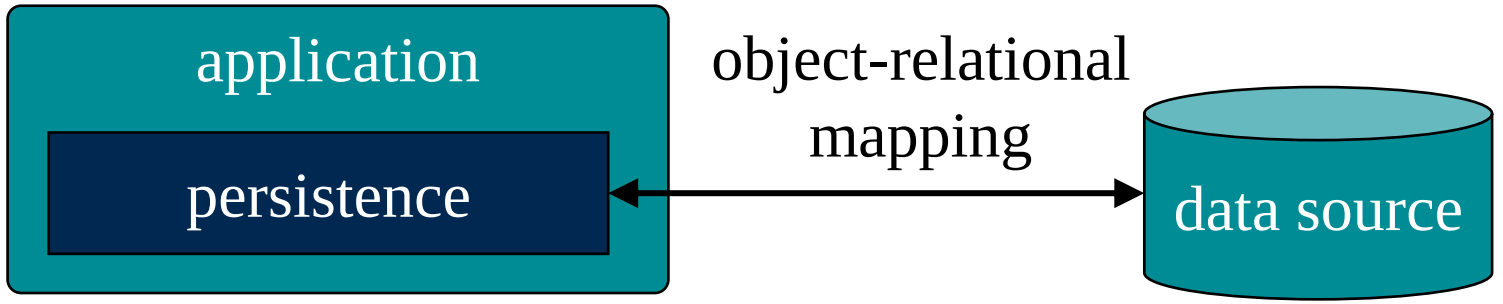
This mapping is easily translatable to an object-oriented environment, rows are objects, tables are classes



OBJECT-RELATIONAL MAPPING

The mapping is called *object-relational mapping* (*ORM*)

- provides a high-level transformation of the database that is easy to use in the program
- also controls the way the data is handled
- the created entity classes are responsible for storing data, not performing operations on them



ENTITY FRAMEWORK

Entity Framework Core implements a platform-independent, complex, object-relational representation of data

- typically, the object-oriented representation of table is an *entity type* (class), and for a row is an *entity object*, but this can be customized
- supports relationships between entities (association, inheritance)
- supports language-integrated queries (LINQ), dynamic data loading, asynchronous data handling
- requires the **`Microsoft.EntityFrameworkCore`** NuGet package and a provider specific package
    - e.g. **`Microsoft.EntityFrameworkCore.SqlServer`** for MSSQL
    - namespace is **`Microsoft.EntityFrameworkCore`**

ENTITY FRAMEWORK

- the advantage of its modular architecture is to load only the necessary components and providers

Database engine	Entity Framework Core NuGet package
MSSQL	Microsoft.EntityFrameworkCore.SqlServer
SQLite	Microsoft.EntityFrameworkCore.Sqlite
MySQL / MariaDB	MySql.Data.EntityFrameworkCore Pomelo.EntityFrameworkCore.MySql
Oracle	Oracle.EntityFrameworkCore
PostgreSQL	Npgsql.EntityFrameworkCore.PostgreSQL

ENTITY FRAMEWORK

There are 3 approaches for defining the model:

- *Code first*: Define the model as code and generate the database schema based on it. Good for new projects.
- *Database first*: Maps the database structure to the entity model, the code is generated based on the database schema. Good for projects with a large pre-existing database.
- *Model first*: Manually build the model with modelling tool and set up the relationships. The database schema and the code is generated from the model.

Changes to the model and database schema can be synchronized.

LOADING RELATED ENTITIES

Entities do not store related data (available through *navigational properties*), but they can be loaded in several ways:

- Eager loading can be done by adding the necessary **`Include()`** procedure calls when constructing the LINQ query. Then, the related entities will be loaded together with the original query.
- Explicit loading can be done to retrieve the related entities of an already loaded entity object, using the **`Entry()`** method of the **`DbContext`** class.
- Lazy loading loads the related entities when they are needed (when they are first evaluated). For this, we need to install the **`Microsoft.EntityFrameworkCore.Proxies`** NuGet package and enable lazy loading (**`UseLazyLoadingProxies()`**) for out database context.

EXAMPLE

Task: Create a travel agency website where you can browse apartments.

- Use a layout (**`_Layout.cshtml`**) for the header and the list of cities to avoid code redundancy
  - the **`Index`** and **`Details`** view should only contain the different page content
- Create two new actions for displaying images
  - one to display an image for a building (**`ImageForBuilding`**),
  - one to load a certain image in either small or large size (**`Image`**)
  - there might be no image, return a default one (**`NoImage.png`**)

EXAMPLE

- Show the location of the buildings on a map
  - the coordinates of the buildings are stored in the buildings table (**`LocationX`**, **`LocationY`**), we just need to pass them to the map in text form (making sure to the decimal separator is a dot)
  - the Google map is displayed on the **`Details`** view, specifying the size of the map and the displayed coordinate
    - The Google Maps JavaScript API is used to display the map.
  - In the configuration (**`appsettings.json`**), we specify API key for Google Maps.
    - One can be obtained from: http://console.developers.google.com

EXAMPLE

Design (*entity model*):

- the **`City`** entity stores the data of the cities it contains
- the **`Building`** entity holds the building data, including the city identifier
- the **`Apartment`** entity stores apartment data, including the building identifier
- the **`BuildingImage`** entity stores the images of the buildings, one larger and one smaller version of each image, and the building identifier

EXAMPLE

Design (*database*):

